ASSIGNMENT - 5

Functions — Calling, Returning, Parameters, Scope, By Value and By Reference, Arrays and Strings in Functions, Library Functions, Macros and Storage Classes — IIITA Campus Life Problems

Course	Problem Solving with Programming (C)
Program	B.TECH — 2026 Batch
Section	IT-SEC A
Assignment No.	5
Subject Area	Functions: definition and call, return value, parameters, prototype, scope, parameter passing by value and by reference, passing arrays, library and string functions, header files, macros, storage classes
Theme	Real IIITA Campus Locations & Situations
Total Programs	10 (each with code, output and line-by-line explanation)

INSTRUCTIONS

Read the full PDF very carefully. Part A explains what a function is and every function concept of this assignment one by one, with a real IIITA example on the left and the matching C code on the right. Part B contains the 10 practice programs based on those concepts.

Today's Class Task

1. Run all **10 programs** given in Part B on your laptop or college system through VS Code.
2. Take a screenshot of the **code and output** for every question.
3. Paste all 10 screenshots into **one Google Doc**, in order from Question 1 to Question 10.
4. Name that Google Doc using your own **Enrollment Number**.
5. Submit it on **Google Classroom** by clicking the **'Turn In'** button.

Note — the comment lines inside the programs

Lines written as `// …` or `/* … */` inside a program are **comments**. The compiler ignores them completely, so they never change the output. They are written for the programmer: to keep the logic clear, and to remember why a line was written. Keep such comments in your own submitted programs as well.

Table of Contents

Part A — Concepts (with real IIITA examples)

Part B — Practice Programming Questions (10 Programs)

PART A — CONCEPTS

Until now every program you have written lived inside one single `main()`. That works while a program is small. But when the same job has to be done in three different places, the same lines have to be typed three times — and if the rule ever changes, all three copies have to be corrected, and one of them always gets forgotten.

A **function** is a named block of work that is written **once** and can then be used from anywhere, as many times as needed. That is the single idea behind this assignment; everything else is built on top of it.

Concept 1 explains it with a counter you have all stood in front of, before any code appears.

What this assignment expects you to know already

Nothing about functions is assumed — it is built up from zero in Part A. Only the tools from your earlier assignments come back, and each one is named here so you can look it up if it has slipped from memory.

TOOL	LEARNT IN	WHERE IT IS USED AGAIN HERE
`int, char, float, printf, scanf`	Assignment 1	everywhere
`if and else`	Assignment 2	Q8
`for and while loops`	Assignment 3	Q6
`arrays and the index`	Assignment 4	Q6, and every string question
`the address operator &`	Assignment 4	Q5, for passing by reference
`character arrays and '\0'`	Assignment 4	Q8

Everything else — the function, the call, the return, the prototype, scope, macros and storage classes — is new, and is explained from the beginning in the concepts that follow.

CONCEPT 1

1. What is a Function?

REAL IIITA SITUATION — THE PHOTOCOPY COUNTER AT CC3

You walk up to the **photocopy counter** with five pages. You do not open the machine, you do not learn how the roller works. You simply **hand the pages over**, wait a moment, and **get the copies back**.

Three things happen there, and they are exactly the three parts of a function:

- you **give something** — the pages
- the counter **does a fixed job** — copying
- you **get something back** — the copies

The whole machine and its work sit behind one short name: "photocopy". Everyone in the queue uses that same counter, and nobody has to build their own machine.

THE SAME IDEA IN C

```c
// the counter, written once
int guestBill(int plates) {
    return plates * 70;
}

// anyone can use it, any number of times
total = guestBill(3);
today = guestBill(5);
```

The rule "Rs. 70 per plate" is written in **one place only**. If the rate changes tomorrow, one line is corrected and every user of the counter is corrected with it.

The counter, drawn

YOU, AT THE COUNTER

```
total = guestBill(3);
```

you hand over 3, and wait

THE COUNTER, DOING ITS FIXED JOB

```
int guestBill(int plates) { return plates * 70; }
```

3 arrives as plates, 210 is handed back

One name, one input, one answer — and the rule Rs. 70 written inside, once.

The three parts, side by side

AT THE PHOTOCOPY COUNTER	→	IN THE PROGRAM
The counter itself, with its fixed job	→	`the function definition`
The short name everyone uses	→	`the function name  guestBill`
The pages you hand over	→	`the parameter  int plates`
Standing at the counter and asking for the job	→	`the call  guestBill(3)`
The copies handed back to you	→	`the return value  return plates * 70;`

The same job with and without a function

WITHOUT A FUNCTION — THE RULE IS WRITTEN THREE TIMES

```
bill1 = 3 * 70;
bill2 = 5 * 70;
bill3 = 2 * 70;
// if the rate changes, three lines
// must be found and corrected
```

WITH A FUNCTION — THE RULE LIVES IN ONE PLACE

```
bill1 = guestBill(3);
bill2 = guestBill(5);
bill3 = guestBill(2);
// if the rate changes, only the
// one line inside the function
```

Four words to fix in your mind

Definition = writing the counter and its work.
Call = standing at the counter and asking for the job to be done.
Parameter = what you hand over.
Return value = what comes back to you.

CONCEPT 2

2. Calling a Function, and What Happens at That Moment

REAL IIITA SITUATION — YOU STOP, THE COUNTER WORKS, YOU CONTINUE

While the counter is copying your pages, **you wait**. You do not walk away and come back later; you stand there until the copies are in your hand, and only then do you continue to your class.

A function call works exactly like that. When `main` calls a function:

1. `main` **pauses** at that line
2. the function runs, from its first line to its last
3. control comes **back to the same line** in main
4. main continues from there

THE ORDER IN WHICH THE LINES REALLY RUN

```
void printMenu() {
    printf("Poha\n");       // step 2
    printf("Rice\n");       // step 3
}

int main() {
    printf("Notice\n");     // step 1
    printMenu();             // the call
    printf("Done");         // step 4
}
```

AT THE PHOTOCOPY COUNTER	→	IN THE CODE
Standing at the counter and asking	→	`printMenu();`
Waiting while the job is done	→	`main pauses at that line`
The job itself	→	`the lines inside the function`
Walking on to your class	→	`main continues at the next line`

Step by step — where control goes

`main, line 1`	→	"Notice" is printed, and main moves to the next line
`printMenu();`	→	**main pauses here and control jumps into the function**
`inside the function`	→	"Poha" and then "Rice" are printed
`the closing brace`	→	**the function is finished, so control returns to the call line in main**
`main, next line`	→	"Done" is printed, exactly where main had paused

Two things beginners expect wrongly

1. Writing a function does not run it. The lines inside a function run only when the function is **called**. A program can contain a perfectly good function that never runs, simply because nobody called it.

2. The call needs the brackets. `printMenu();` calls it. Writing `printMenu;` without the brackets does not call anything at all.

CONCEPT 3

3. Parameters — Handing Values Over to the Function

REAL IIITA SITUATION — HOW MANY PLATES?

At the mess guest counter the rule is the same for everybody: **Rs. 70 per plate**. What changes from person to person is only one thing — **how many plates**.

So the counter needs one piece of information from you before it can do anything. That piece of information is a **parameter**.

The value you actually say at the counter — 3 plates — is called the **argument**. The name written inside the function, `plates`, is the **parameter**. The argument is what is given; the parameter is where it lands.

PARAMETER AND ARGUMENT

```
// plates is the PARAMETER
int guestBill(int plates) {
    return plates * 70;
}

// 3 is the ARGUMENT
total = guestBill(3);
```

More than one value can be handed over. They are separated by commas, and they must arrive in the **same order** as the parameters are written.

THE CALL, IN MAIN

```
total = guestBill(3);
```

THE FUNCTION THAT RECEIVES IT

```
int guestBill(int plates) { … }
```

The value 3 travels from the call into the parameter `plates`, and the function then works with that name.

Two parameters, in order

AT THE BADMINTON DESK	→	IN THE CODE
"There are 8 slots today"	→	the first argument `slotsLeft(8, 5)`
"5 of them are already booked"	→	the second argument
The clerk's own two words for them	→	`int total, int booked`
Saying the two numbers the wrong way round	→	`slotsLeft(5, 8)` — a different, wrong answer

The order is everything

C does not check whether the numbers make sense; it only fills the parameters from left to right. `slotsLeft(8, 5)` gives 3, while `slotsLeft(5, 8)` gives -3 — the compiler will not complain about either. The programmer is the only guard here, exactly as with array boundaries in Assignment 4.

CONCEPT 4

4. The return Statement and the Return Value

REAL IIITA SITUATION — WHAT COMES BACK ACROSS THE COUNTER

Some counters hand something back, and some do not.

The **photocopy counter** gives you copies — something comes back, and you carry it away.

The **notice board** is different. It shows the menu to everyone, but it does not hand you anything to carry. The work is done, yet nothing comes back.

C has both kinds. The word written **before** the function name says which kind it is: a type such as `int` means something comes back, and `void` means nothing does.

THE TWO KINDS

```c
int guestBill(int plates) {
    return plates * 70;   // hands one back
}
```

```c
void printMenu() {
    printf("Poha\n");       // does the job,
}                          // nothing comes back
```

WRITTEN AS	MEANING	HOW IT IS USED IN MAIN
`int guestBill(...)`	an int comes back	`total = guestBill(3);`
`float average(...)`	a decimal number comes back	`avg = average(marks, 5);`
`void printMenu()`	nothing comes back	`printMenu();  // used alone`

AT THE TWO COUNTERS	→	IN THE CODE
The photocopy counter, which hands something back	→	`int guestBill(...) with return`
The notice board, which hands nothing back	→	`void printMenu() with no return`
Carrying the copies away	→	`total = guestBill(3);`
Just reading the board and walking on	→	`printMenu();`

What return really does

`return plates * 70;`	→	the value is worked out first: 3 × 70 = 210
`the same line`	→	**210 is handed back, and the function ends immediately**
`back in main`	→	the call `guestBill(3)` now stands for the value 210
`total = ...;`	→	210 is stored in total, and main carries on

Three rules about return

1. It ends the function at once. Any line written after a `return` that has already run will never run.

2. Only one value can come back. A function cannot return two answers with one return. When two values must change, the address is sent instead — that is Concept 8.

3. A void function needs no return. It may still contain a bare `return;` to finish early, but it must never return a value.

CONCEPT 5

5. The Function Prototype — the Promise Written at the Top

REAL IIITA SITUATION — THE BOARD ABOVE THE COUNTER

Above the photocopy counter hangs a small board: **"Photocopy — bring pages, get copies, Rs. 2 per page."**

The board is not the machine. It only tells everyone, in advance, **what the counter accepts and what it gives back**, so that people can join the queue correctly even before they see the machine.

A **prototype** is that board. It is one line written near the top of the program that announces the function's name, what it takes and what it returns. The real function can then be written lower down, after `main`.

C reads a program from top to bottom. Without the board, when the compiler reaches the call inside `main`, it has never heard of that function and refuses to continue.

PROTOTYPE, CALL, DEFINITION

```c
// 1. the promise
int slotsLeft(int total, int booked);

int main() {
    // 2. the call, allowed now
    left = slotsLeft(8, 5);
}

// 3. the real work, written later
int slotsLeft(int total, int booked) {
    return total - booked;
}
```

PART	WHAT IT IS	ENDS WITH
`int slotsLeft(int total, int booked);`	the **prototype** — only the promise	a **semicolon**
`int slotsLeft(int total, int booked) { … }`	the **definition** — the actual work	a **block** in braces
`slotsLeft(8, 5)`	the **call** — asking for the job	the values handed over

AT THE PHOTOCOPY COUNTER	→	IN THE CODE
The board announcing the job in advance	→	`the prototype, ending with ;`
The machine that actually does the work	→	`the definition, with a body in { }`
Standing in the queue and asking	→	`the call, inside main`

Step by step — how the compiler reads the file

`line 3`	→	the prototype is read: "a function named slotsLeft exists, it takes two ints and gives back an int"
`inside main`	→	**the call is met — and it is allowed, because the promise was already read**
`below main`	→	the real definition is read, and it matches the promise
`without the prototype`	→	the call would be met first, with the function still unknown, and the compiler would stop there

When is a prototype needed?

Not needed when the whole function is written **above** `main`, because the compiler has already read it by the time it meets the call.

Needed when the function is written **below** `main`, which is the common style in larger programs, because it keeps `main` — the story of the program — at the top where a reader looks first.

The semicolon is the whole difference

A prototype ends with `;` and stops there. Forgetting that semicolon turns the line into the start of a definition, and the compiler then complains about the very next line instead — which is why this error often looks confusing.

CONCEPT 6

6. Local Variables, Local Scope and Global Scope

REAL IIITA SITUATION — INSIDE THE COUNTER, AND ON THE NOTICE BOARD

The rough paper that the counter clerk scribbles his totals on stays **behind the counter**. Nobody in the queue can see it, and when he finishes with your job he throws it away. The next customer's clerk starts with a fresh sheet.

The **hostel notice board** is the opposite. It hangs in the corridor where everyone can read it, and it stays there all day.

A variable declared **inside** a function is like the rough paper: it is **local**. It is created when the function starts, it cannot be seen from anywhere else, and it disappears when the function ends.

A variable declared **outside every function** is like the notice board: it is **global**, and every function in the file can read and change it.

LOCAL AND GLOBAL

```
int messRate = 70;      // GLOBAL

int guestBill(int plates) {
    int bill;           // LOCAL to this one
    bill = plates * messRate;
    return bill;
}

int main() {
    // bill cannot be used here at all
    printf("%d", messRate);  // allowed
}
```

AT THE COUNTER AND IN THE CORRIDOR	→	IN THE CODE
The clerk's rough sheet, thrown away after	→	a local variable, declared inside a function
The corridor notice board everyone reads	→	a global variable, declared outside every function
Two clerks writing "count" on their own sheets	→	two locals with the same name, in two functions
The rate painted on the notice board	→	int messRate = 70; above every function

Who can see what

The green box is readable everywhere. The orange box exists only while `guestBill` is running, and main can never touch it.

	LOCAL VARIABLE	GLOBAL VARIABLE
Declared	inside a function	outside every function, usually at the top
Who can see it	only that one function	every function in the file
Born	when the function is called	when the program starts
Dies	when the function ends	when the program ends
Starting value	rubbish, until you give it one	0, given automatically by C

Why two functions may use the same name safely

TWO DIFFERENT VARIABLES, ONE NAME

```
void deskA() {
    int count = 5;    // deskA's own count
}

void deskB() {
    int count = 9;    // a different count
}
```

WHY THIS IS SAFE

Each clerk has his own rough sheet, so both can write "count" on their own sheet without disturbing each other. Changing one does not touch the other.

This is one of the real reasons functions are useful: you can write a function without first checking what variable names the rest of the program happens to use.

Use global variables sparingly

Because **any** function can change a global variable, a wrong value can come from anywhere in the file, and finding out who changed it is hard work. Prefer passing values in as parameters and handing the answer back with `return`; keep globals for things that are truly shared, such as a fixed rate.

CONCEPT 7

7. Parameter Passing by Value — the Copy Goes, the Original Stays

REAL IIITA SITUATION — YOU HAND OVER A PHOTOCOPY, NOT THE ORIGINAL

When you submit a document at the office, you hand in a **photocopy** and keep the original in your file. If the clerk writes all over the copy, stamps it, or tears it, **your original is untouched**.

This is exactly what C does by default. When you call `tryToSwap(first, second)`, the function does not receive your variables. It receives **copies of their values**.

So anything the function does to them changes only the copies, which are thrown away when the function ends. This is called **passing by value**.

THE SWAP THAT DOES NOT SWAP

```
void tryToSwap(int a, int b) {
    int temp = a;
    a = b;
    b = temp;        // only copies swapped
}

tryToSwap(first, second);
// first and second are unchanged
```

AT THE OFFICE COUNTER	→	IN THE CODE
Handing over a photocopy	→	`tryToSwap(first, second)`
The clerk's own two sheets	→	`int a, int b`
Him rearranging his own sheets	→	`a = b; b = temp;`
Your originals, untouched in your file	→	`first and second in main`

What the two sides hold, moment by moment

`before the call`	→	main: first = 11, second = 22
`at the call`	→	**copies are made: a = 11, b = 22 — two new variables inside the function**
`inside`	→	the copies are swapped: a = 22, b = 11
`function ends`	→	a and b are destroyed; nothing was sent back
`after the call`	→	main: first = 11, second = 22 — exactly as before

This is not a bug — it is protection

Passing by value means a function can never damage the caller's variables by accident. It is the safe default, and it is what you want almost every time. Only when the caller's variable really **must** change do you send the address instead, which is the next concept.

CONCEPT 8

8. Parameter Passing by Reference — Sending the Address

REAL IIITA SITUATION — TELLING THE OFFICE WHICH LOCKER TO OPEN

Sometimes a copy is not enough. If the office has to **correct the record itself**, handing over a photocopy is useless — they need to reach the original.

So instead of a copy you tell them **where the original is kept**: locker number 12 in room 5042. Now they can open that exact locker and change what is inside.

In C the "locker number" is the **address**, written with & — the same address operator you used for `scanf` in Assignment 4. The function receives the address in a **pointer** parameter, written with a star: `int *a`.

Inside the function, *a means "the value kept at that address". Writing to *a changes the caller's own variable.

THE SWAP THAT REALLY SWAPS

```
void reallySwap(int *a, int *b) {
    int temp = *a;   // value at address a
    *a = *b;
    *b = temp;
}

reallySwap(&first, &second);
// first and second really change
```

	BY VALUE (CONCEPT 7)	BY REFERENCE (THIS CONCEPT)
What is handed over	a copy of the value	the address of the original
Written in the call	`tryToSwap(first, second)`	`reallySwap(&first, &second)`
Written in the function	`int a`	`int *a`
Reading the value inside	`a`	`*a`
Can the original change?	**no**	**yes**
Use it when	the function only needs to read the value	the function must change the caller's variable, or must change more than one

AT THE OFFICE	→	IN THE CODE
Telling them which locker holds the record	→	`&first`
The clerk writing that locker number down	→	`int *a`
Opening the locker and reading it	→	`*a`
Putting a new value into that locker	→	`*a = *b;`

Step by step, with the addresses shown

The function does not get 11 and 22. It gets 2000 and 2004 — the two locker numbers — so *a reaches straight back into main's own box.

Two stars, two meanings

In the declaration `int *a` the star says "this parameter holds an address". In the line `*a = *b;` the star says "the value living at that address". Same symbol, two jobs — and reading the line aloud as "the value at a becomes the value at b" keeps it clear.

CONCEPT 9

9. Passing an Array to a Function

REAL IIITA SITUATION — YOU DO NOT CARRY THE WHOLE SHELF

If the office needs the marks of the whole class, nobody photocopies sixty sheets and carries them across. They are told **where the register is kept**, and they walk to it.

C does the same with arrays, and it does it automatically. When an array is passed to a function, **the address of its first box is sent**, never a copy of all the values.

Two things follow from that, and both matter:

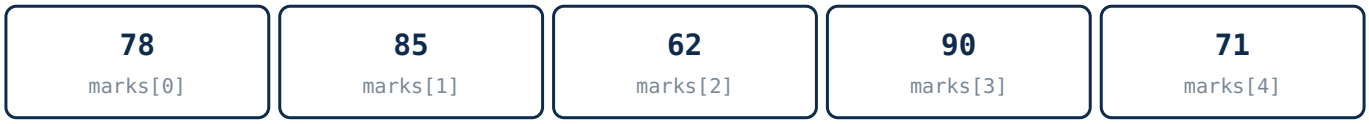
- no & is written in the call — the array name already **is** the address (Assignment 4, Concept 6)
- changes made inside the function **do** affect the caller's array, because there is only one register

The function is also not told how long the array is, so the **size must be passed as a second parameter**.

PASSING AN ARRAY AND ITS SIZE

```
int total(int marks[], int n) {
    int i, sum = 0;
    for (i = 0; i < n; i++) {
        sum = sum + marks[i];
    }
    return sum;
}

sum = total(marks, 5);   // no & here
```



There is only **one** row of boxes in the whole program. The function is told where it begins, so it reads main's own boxes.

AT THE OFFICE, WITH THE REGISTER	→	IN THE CODE
Telling them where the register is kept	→	`total(marks, 5)`
The clerk's own word for that register	→	`int marks[]`
Telling them how many rows it has	→	`int n`
Reading row i of the register	→	`marks[i]`
There is only one register, not a copy	→	`changes inside the function are real`

Why the size must travel with the array

Inside the function, `sizeof(marks)` does **not** give 20 the way it did in Assignment 4. The function holds only an address, so it gives the size of an address instead — 8 on most machines today, and 4 on older ones. The function therefore has no way of knowing where the array ends, which is why passing n is not optional: without it, the loop would run past the last box.

CONCEPT 10

10. One Function Calling Another — Nested Calls

REAL IIITA SITUATION — THE COUNTER THAT SENDS YOU TO ANOTHER COUNTER

You ask the mess billing clerk for the total. To work it out he himself turns to the **rate counter** and asks for the plate cost, then adds the rice charge to it, and only then hands you the final bill.

You never spoke to the rate counter. You asked one clerk; **he asked another**.

A function may call another function in exactly the same way. The rule from Concept 2 simply applies twice: the caller pauses, the called function runs completely, and control comes back.

A FUNCTION CALLING A FUNCTION

```
int plateCost(int plates) {
    return plates * 70;
}

int billWithRice(int plates, int rice) {
    int cost = plateCost(plates);   // inner
    return cost + rice * 10;
}

bill = billWithRice(3, 2);   // outer
```

AT THE MESS COUNTERS	→	IN THE CODE
You asking the billing clerk	→	`billWithRice(3, 2)`
He asking the rate counter	→	`plateCost(plates)`
The rate counter answering first	→	`return plates * 70;`
The billing clerk finishing after him	→	`return cost;`

Step by step — three levels deep and back

`main`	→	calls billWithRice(3, 2) and pauses
`billWithRice`	→	**calls plateCost(3) and pauses in its turn**
`plateCost`	→	works out 3 × 70 = 210 and returns it
`billWithRice`	→	continues where it paused: 210 + 2 × 10 = 230, and returns 230
`main`	→	continues where it paused, with bill = 230

The last one called is the first one finished

Notice the order in which the functions ended: **plateCost first, then billWithRice, then main** — the reverse of the order in which they started. A function always finishes before the one that called it can continue. Concept 15 shows why the computer never loses track of this.

CONCEPT 11

11. Library Functions and Header Files

REAL IIITA SITUATION — THE COUNTERS THAT WERE ALREADY THERE

Nobody at IIITA built the photocopy machine, the water cooler or the printer. They were **already installed**, and you simply use them.

C works the same way. Hundreds of functions are already written and shipped with the compiler — these are the **library functions**. You have used two of them since your very first program: `printf` and `scanf`.

But the compiler still needs the **board above the counter** — the prototype from Concept 5 — before it will let you call them. Those prototypes are collected in **header files**, and `#include` brings them in.

That is why `#include <stdio.h>` has been at the top of every program you have written: it is what makes `printf` legal.

BRINGING THE PROTOTYPES IN

```
#include <stdio.h>    // printf, scanf
#include <string.h>   // strlen, strcpy ...
#include <math.h>     // sqrt, pow ...
```

The angle brackets tell the compiler to look in its own folder of standard headers.

WITHOUT THE HEADER

```
strlen(name);
```

the compiler has never heard of strlen and complains

WITH THE HEADER

```
#include <string.h>
strlen(name);
```

the prototype has been read, so the call is accepted

The function itself was always there in the library. The header only tells the compiler what it looks like.

ON THE CAMPUS	→	IN THE CODE
The machines already installed	→	`the library functions`
The board saying what a machine accepts	→	`the prototype inside a header file`
Fetching that board before using the machine	→	`#include <stdio.h>`
Using the machine	→	`printf("...");`

HEADER FILE	WHAT IT BRINGS IN	FUNCTIONS YOU WILL MEET
`stdio.h`	standard input and output	`printf, scanf`
`string.h`	work on character arrays	`strlen, strcpy, strcat, strcmp`
`math.h`	mathematical work	`sqrt, pow, abs`

Two practical points

1. A missing header is not a missing function. The function is still there in the library; the compiler simply has not been told what it looks like, so it complains at the call. The cure is the right `#include`, not rewriting the function.

2. math.h sometimes needs one extra word on the command line. On Linux, programs using `sqrt` are compiled with `gcc file.c -lm`. Inside VS Code on Windows this is usually not needed.

CONCEPT 12

12. The Common String Library Functions

REAL IIITA SITUATION — THE FOUR JOBS AT THE REGISTER DESK

At the Auditorium register the clerk does the same four small jobs with names all day: **count the letters**, **copy a name into another column**, **join two names together**, and **check whether two names are the same**.

In Assignment 4 you did the first of these by hand, walking to the `'\0'` mark. C already has a counter for each of these four jobs, and they all live in `string.h`.

They all stop at the same `'\0'` end mark you learnt in Assignment 4 — nothing new is happening underneath.

THE FOUR IN USE

```c
#include <string.h>

char a[20] = "Subrata";
char b[20] = "Pramanik";
char full[40];

strlen(a);            // 7
strcpy(full, a);      // full = "Subrata"
strcat(full, b);      // joins b at the end
strcmp(a, b);         // not 0, so different
```

FUNCTION	WHAT IT DOES	WHAT IT GIVES BACK	CAREFUL ABOUT
`strlen(s)`	counts the letters up to `'\0'`	the count, as a number	the end mark itself is **not** counted
`strcpy(dest, src)`	copies src into dest	—	dest must be large enough, or the copy runs past its last box
`strcat(dest, src)`	joins src onto the end of dest	—	dest must have room for **both** names and the end mark
`strcmp(a, b)`	compares two strings letter by letter	**0** when they are the same	0 means equal — the opposite of what most people expect

AT THE REGISTER DESK	→	IN THE CODE
Counting the letters of a name	→	`strlen(a)`
Copying a name into another column	→	`strcpy(full, a);`
Joining the second name onto it	→	`strcat(full, b);`
Checking whether two names are the same	→	`strcmp(a, b) == 0`

What strcpy and strcat really do to the boxes

After `strcpy(full, a)` — seven letters and the end mark are copied into full

After `strcat(full, " ")` and `strcat(full, b)` — the joining starts exactly where the old end mark was, and a new end mark is written after the last letter

The mistake that strcmp causes

Write `if (strcmp(a, b) == 0)` to mean **"the names are the same"**. Writing `if (strcmp(a, b))` means "the answer is not zero", which is true when they are **different** — the exact opposite of what was intended.

CONCEPT 13

13. Macro Definition — #define

REAL IIITA SITUATION — THE RATE WRITTEN ONCE ON THE BOARD

The mess rate, Rs. 70, is written on one board at the entrance. Every clerk reads that board instead of remembering his own number. When the rate changes, **one board** is repainted.

A **macro** is that board. `#define PI 3.14159` tells the compiler: before compiling anything, go through the whole program and **replace every PI with 3.14159**.

This happens **before** compiling, and it is a plain text replacement. A macro is therefore **not a variable**: it has no type, it takes no memory, and it can never be changed while the program runs.

A MACRO AND WHAT IT BECOMES

```
#define PI 3.14159      // no ; at the end
#define SIDE 5

area = PI * 10 * 10;
```

```
// what the compiler actually sees
area = 3.14159 * 10 * 10;
```

AT THE MESS ENTRANCE	→	IN THE CODE
The rate painted on one board	→	`#define PI 3.14159`
Every clerk reading that board	→	`every PI in the program`
Repainting the board once	→	`changing the single #define line`
The board is not a person and cannot change its mind	→	`a macro takes no memory and never changes`

PI	3.14159	SIDE	5
what you write	what the compiler sees	what you write	what the compiler sees

Plain text replacement, done before compiling — which is why a macro has no type and takes no memory

	MACRO `#define PI 3.14159`	VARIABLE `float PI = 3.14159;`
When it acts	before compiling, as text replacement	while the program runs
Memory used	none	a box in memory
Can it change?	never	yes, any line can change it
Ends with a semicolon?	**no**	yes

The semicolon trap

Writing `#define PI 3.14159;` puts the semicolon **into** the replacement. Then `area = PI * 10;` becomes `area = 3.14159; * 10;` — and the error the compiler reports points at a line that looks perfectly correct. A `#define` line never ends with a semicolon.

CONCEPT 14

14. Storage Classes — auto, static, register, extern

A storage class answers two questions about a variable: **how long does it live**, and **who can see it**. You have already met the answers without the names — Concept 6 described exactly this for local and global variables. There are four words in all.

REAL IIITA SITUATION — FOUR KINDS OF RECORD AT THE GATE

auto — the rough sheet the guard uses for one visitor and then throws away. Every ordinary local variable is this by default.

static — the register that stays in the drawer. The guard takes it out for every visitor, adds a line, and puts it back. It **remembers** what was written before.

register — the number the guard simply keeps in his head because he needs it constantly. Fast, but there is room for very few.

extern — a register kept in **another office**. The guard does not own it; he only declares that it exists elsewhere, and uses it.

THE FOUR WORDS IN ONE FUNCTION

```c
void enterGate() {
    int today = 0;        // auto by default
    static int count = 0;   // remembered
    register int i;     // keep it handy

    count++;
    today++;

    for (i = 0; i < 3; i++) {  // i is used
        printf(".");
    }
}

extern int messRate;   // lives elsewhere
```

STORAGE CLASS	LIVES	SEEN BY	STARTING VALUE
`auto`	only while the function runs	that function only	rubbish, until you set it
`static`	the whole program, but created only once	that function only	0, given by C
`register`	only while the function runs	that function only	rubbish; the address operator & cannot be used on it
`extern`	the whole program	every file that declares it	0, given by C

AT THE GATE	→	IN THE CODE
The rough sheet for one visitor	→	`int today; (auto)`
The register kept in the drawer	→	`static int count;`
A number kept in the head	→	`register int i;`
A register belonging to another office	→	`extern int messRate;`

Step by step — why static is different, over three calls

`call 1`	→	today is created and becomes 1; count is created once and becomes 1
`call 2`	→	**today is created fresh and becomes 1 again; count is still there and becomes 2**
`call 3`	→	**today is 1 once more; count becomes 3**
`the difference`	→	a plain local variable starts again at every call; a static one is created once and keeps its value

static is not global

A static local variable **lives** as long as a global does, but it is still **visible only inside its own function**. That is exactly why it is useful: it remembers, without letting the rest of the program reach it.

CONCEPT 15

15. How a Function Call Really Works, and Why Functions are Used

REAL IIITA SITUATION — THE PILE OF SLIPS AT THE COUNTER

When the billing clerk turns to the rate counter, he puts his own half-finished slip **on top of a pile** so that he can pick it up again the moment he gets his answer.

The computer does the same thing. At every call it puts a small record on a pile: the **parameters**, the function's own **local variables**, and the **line to come back to**.

When the function returns, its record is thrown off the pile, and the record underneath — the caller's — becomes the top one again. That is why the caller continues at exactly the right line, and why the **last function called is always the first to finish**.

It is also why local variables disappear at the end of a call: their record went off the pile with it.

THE PILE DURING Q7

`plateCost` ← running now, on top

`billWithRice` waiting

`main` waiting, at the bottom

As each one returns, the top record is removed and the one below carries on.

AT THE COUNTERS	→	IN THE CODE
Putting your half-finished slip on the pile	→	`the record made at every call`
The slip on top being worked on now	→	`the function currently running`
Throwing the top slip away when done	→	`the record removed at return`
Picking up the slip underneath	→	`the caller continuing at the right line`

Why functions are used — and where they cost something

ADVANTAGE	WHAT IT MEANS IN PRACTICE	SEEN IN
The rule is written once	The rate Rs. 70 lives in one line. Change it there and every caller is corrected at once.	Concept 1, Q2
The program becomes readable	`main` reads like the story of the work, not like hundreds of lines of detail.	Q7
The same job serves many callers	One function, called with 3 plates, then 5, then 2.	Concept 1
Names cannot clash	Local variables are private, so a function can be written without checking the rest of the file.	Concept 6
Ready-made work is free	strlen, strcpy, sqrt are already written, tested and fast.	Concepts 11 and 12, Q8, Q9

COST OR LIMITATION	WHAT HAPPENS, AND WHAT TO DO ABOUT IT	SEEN IN
Only one value can be returned	When two values must come back, addresses are passed instead.	Concepts 4 and 8, Q5
Copies are made at every call	Passing by value copies the value. For a single number this costs nothing worth counting; for very large data an address is sent instead.	Concepts 7 and 9
An array loses its length	The function receives only an address, so `sizeof` no longer gives the total size and the size must be passed separately.	Concept 9, Q6
Each call costs a little time	Putting the record on the pile and taking it off is not free. It matters only inside code that runs millions of times.	this concept
Too many globals undo the benefit	If functions share their data through globals instead of parameters, the isolation that makes functions safe is lost.	Concept 6

How to decide, in one sentence

Write a function whenever a job has a **clear name**, needs a **small number of inputs**, and produces **one clear result** — and pass what it needs as parameters rather than letting it reach out to global variables.

PART B — PRACTICE PROGRAMMING QUESTIONS

Each of the 10 programs below is built on a concept from Part A, and is given with its real IIITA situation, the complete program, a line-by-line explanation, and the exact output.

Which concept each question uses

QUESTION	CONCEPT USED
Q1 — Mess notice board	a function with no parameter and no return value (Concepts 1, 2 and 4)
Q2 — Guest meal bill	a parameter going in, a value coming back (Concepts 3 and 4)
Q3 — Badminton slots	the prototype above main, the definition below it (Concept 5)
Q4 — Swap that fails	parameter passing by value (Concept 7)
Q5 — Swap that works	parameter passing by reference (Concept 8)
Q6 — Total of the marks	passing an array and its size (Concept 9)
Q7 — Bill with rice	one function calling another (Concept 10)
Q8 — Names at the register	the string library functions (Concepts 11 and 12)
Q9 — Campus measurements	macros and a library function (Concepts 11 and 13)
Q10 — Gate register	a static variable against a plain one (Concept 14)

QUESTION 1 — A FUNCTION WITH NO INPUT AND NO RETURN

Q1. Hostel Mess Notice Board

REAL SITUATION

Morning — the notice board at the hostel mess.

The same menu has to be shown again and again: at breakfast, at lunch and at dinner. The board does exactly one job — it **shows the menu** — and it does not need anything from you, nor does it hand you anything to carry away.

So the function needs **no parameter** and returns **no value**. It is written once and can be called from anywhere.

CONCEPT USED

A **void** function with an empty parameter list.

`void` before the name means nothing comes back; the empty brackets mean nothing goes in.

Watch the order of the printed lines: it shows exactly where control travels.

```c
#include <stdio.h>

void printMenu() {                  /* no input, no value returned */
    printf("Today's mess menu\n");
    printf("Breakfast : Poha\n");
    printf("Lunch     : Rice and Dal\n");
    printf("Dinner    : Roti and Sabji\n");
}

int main() {
    printf("Notice board at the hostel mess\n\n");

    printMenu();                    /* the call */

    printf("\nMenu printed by the function");

    return 0;
}
```

AT THE NOTICE BOARD	→	IN THE CODE
The board and the job it does	→	`void printMenu() { ... }`
Nothing has to be handed over to it	→	`the empty brackets ( )`
Nothing is carried away from it	→	`void, and no return statement`
Asking for the menu to be shown	→	`printMenu();`

Step by step — where control goes

`main starts`	→	the heading line is printed, then a blank line from the second `\n`
`printMenu();`	→	**main pauses; control jumps into the function**
`inside`	→	the four menu lines are printed, in the order they are written
`closing brace`	→	**the function ends and control returns to main**
`main continues`	→	the last line is printed, exactly where main had paused

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
`void printMenu()`	The definition. `void` says no value comes back; the empty brackets say no value goes in. It is written **above** main, so no prototype is needed here.
`the four printf lines`	The body — the job being packed into one name. They run only when the function is called, never on their own.
`printf("... \n\n");`	The heading in main. The second `\n` leaves one blank line, so the menu stands apart from the heading.
`printMenu();`	The call. The brackets are what make it a call; without them the line would do nothing.
`printf("\nMenu printed ...");`	Runs after the function has finished, which proves that control really came back to main.
`return 0;`	Belongs to `main`, not to the function. It tells the operating system the program finished successfully.
`#include <stdio.h>`	Brings in the prototype of `printf`, which is itself a library function — the same idea that Concept 11 explains in full.
`int main()`	The program always starts here, whatever else is written above it. The function defined above main did not run until it was called.

OUTPUT

```
Notice board at the hostel mess

Today's mess menu
Breakfast : Poha
Lunch     : Rice and Dal
Dinner    : Roti and Sabji

Menu printed by the function
```

QUESTION 2 — A PARAMETER IN, A VALUE BACK

Q2. BH-5 Guest Meal — a Function that Returns the Bill

REAL SITUATION

Afternoon — the guest meal counter at the BH-5 mess.

The rate is fixed at **Rs. 70 per plate**. What changes from guest to guest is only **how many plates** are ordered. Today three plates are ordered.

So the counter needs one value from you, and it hands one value back — the bill.

CONCEPT USED

A function with a **parameter** and a **return value**.

`int` before the name promises that an int comes back; `int plates` is where the handed-over value lands.

The rate Rs. 70 is written in exactly one line of the whole program.

```c
#include <stdio.h>

int guestBill(int plates) {         /* takes one value, returns one value */
    int bill;
    bill = plates * 70;
    return bill;                    /* the answer goes back to main */
}

int main() {
    int total;

    total = guestBill(3);           /* 3 is handed over */

    printf("Guest plates ordered = 3\n");
    printf("Rate per plate       = Rs. 70\n");
    printf("Total bill           = Rs. %d", total);

    return 0;
}
```

IN MAIN — THE ARGUMENT

```c
total = guestBill(3);
```

IN THE FUNCTION — THE PARAMETER

```c
int guestBill(int plates)
```

3 travels into `plates`; the answer 210 travels back and lands in `total`.

AT THE GUEST COUNTER	→	IN THE CODE
"Three plates, please"	→	`guestBill(3)`
The clerk's own word for that number	→	`int plates`
The fixed rate of Rs. 70 a plate	→	`bill = plates * 70;`
The bill handed across the counter	→	`return bill;`
You putting the bill in your pocket	→	`total = guestBill(3);`

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
`int guestBill(int plates)`	The first `int` promises an int will come back; `int plates` is the parameter that receives the handed-over number.
`int bill;`	A **local** variable. It exists only while the function runs and cannot be used from main at all.
`bill = plates * 70;`	The rule of the counter, written in one place. Changing 70 here changes every bill in the program.
`return bill;`	Hands the value back and ends the function at once. Any line written after it would never run.
`total = guestBill(3);`	The call. After it finishes, `guestBill(3)` stands for 210, and that value is stored in total.
`printf(..., total);`	Prints the value that came back. The function itself printed nothing — it only calculated.
`int total;`	A local variable of main, waiting to receive the value that comes back. It is declared before the call, not after.
`the first two printf lines`	Fixed lines that state the inputs, so the reader can check 3 × 70 by hand against the printed answer.
`return 0;`	Belongs to main. The function has its own `return bill;`, which is a different return doing a different job.

OUTPUT

```
Guest plates ordered = 3
Rate per plate       = Rs. 70
Total bill           = Rs. 210
```

QUESTION 3 — THE PROTOTYPE

Q3. Badminton Court — Using a Prototype Above main

REAL SITUATION

Evening — the booking desk at the badminton court.

There are **8 slots** today and **5** are already booked. The clerk works out how many are still free.

This time the function is written **below** `main`, so that main — the story of the program — is read first. That is exactly when a prototype is needed.

CONCEPT USED

The **prototype**: one line at the top announcing the function, ending with a **semicolon**.

It lets the compiler accept the call in main even though the real function appears much later in the file.

Two parameters are used here, and their **order matters**.

```c
#include <stdio.h>

int slotsLeft(int total, int booked);        /* prototype: the promise */

int main() {
    int left;

    left = slotsLeft(8, 5);

    printf("Badminton court slots today = 8\n");
    printf("Slots already booked        = 5\n");
    printf("Slots still free            = %d", left);

    return 0;
}

int slotsLeft(int total, int booked) {       /* definition: the real work */
    return total - booked;
}
```

AT THE BOOKING DESK	→	IN THE CODE
The board announcing what the desk does	→	`int slotsLeft(int total, int booked);`
"8 slots today, 5 already booked"	→	`slotsLeft(8, 5)`
The clerk's two words for those numbers	→	`int total, int booked`
The subtraction he does	→	`return total - booked;`

Step by step — the three lines that belong together

`line 3`	→	the prototype is read: a function named slotsLeft exists, taking two ints and giving back an int
`inside main`	→	**the call slotsLeft(8, 5) is met — allowed, because the promise was read first**
`during the call`	→	8 lands in total, 5 lands in booked, and 8 - 5 = 3 is returned
`below main`	→	the real definition is read, and it matches the promise word for word

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
`int slotsLeft(int total, int booked);`	The prototype. It ends with a **semicolon** and has no body. Without it, the compiler would meet the call in main having never heard of this function.
`left = slotsLeft(8, 5);`	The call. 8 lands in `total` and 5 in `booked`, strictly left to right. Writing `slotsLeft(5, 8)` would give -3 without any complaint from the compiler.
`int left;`	A local variable of main, waiting to receive the returned value.
`the definition below main`	The same first line as the prototype, but with a **body in braces** and no semicolon. This is where the work actually lives.
`return total - booked;`	The subtraction is worked out first, and the answer is handed straight back. No local variable is needed for it.
`#include <stdio.h>`	Brings in the prototype of `printf` — the same idea as the prototype on the next line, but for a library function.
`the two fixed printf lines`	State the two inputs, 8 and 5, so that the printed answer can be checked by eye.
`return 0;`	Ends main. The function below has its own return, which hands back the subtraction.

OUTPUT

```
Badminton court slots today = 8
Slots already booked        = 5
Slots still free            = 3
```

QUESTION 4 — PASSING BY VALUE

Q4. Passing by Value — Why the Originals Do Not Change

REAL SITUATION

At the office counter, you hand over photocopies.

Two numbers, 11 and 22, are kept in main. A function is asked to exchange them.

The function does exchange them — but it exchanges its **own copies**. When it ends, the copies are thrown away and main's two variables are exactly as they were.

This program prints the values at three moments so that the whole thing can be seen rather than believed.

CONCEPT USED

Parameter passing by value — the default in C.

The function receives copies, so nothing it does can reach main's variables.

This is not a failure of the program; it is the protection that makes functions safe.

```c
#include <stdio.h>

void tryToSwap(int a, int b) {      /* copies arrive here */
    int temp;
    temp = a;
    a = b;
    b = temp;
    printf("Inside the function : a = %d, b = %d\n", a, b);
}

int main() {
    int first = 11;
    int second = 22;

    printf("Before the call     : first = %d, second = %d\n", first, second);

    tryToSwap(first, second);       /* only the values are handed over */

    printf("After the call      : first = %d, second = %d\n", first, second);
    printf("The originals did not change - only the copies were swapped");

    return 0;
}
```

The two sides, moment by moment

11	22	11	22
first (main)	second (main)	a (a copy)	b (a copy)

At the moment of the call: two new boxes are created for the function, holding copies

11	22	22	11
first — untouched	second — untouched	a — swapped	b — swapped

Inside the function: only the two copies changed places. When the function ends, those two boxes are destroyed.

AT THE OFFICE COUNTER	→	IN THE CODE
Handing over a photocopy, keeping the original	→	`tryToSwap(first, second)`
The clerk's own two sheets	→	`int a, int b`
Him rearranging his own sheets	→	`temp = a; a = b; b = temp;`
Your originals, still in your file	→	`first and second, unchanged`

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
`void tryToSwap(int a, int b)`	Two ordinary int parameters, so two **copies** are made at the call. `void` because nothing is sent back.
`int temp; temp = a; ...`	The three-line swap from Assignment 4. It works perfectly — but only on a and b.
`the printf inside`	Proves the swap really happened **inside**, so that the unchanged values in main cannot be blamed on a broken swap.
`tryToSwap(first, second);`	No & is written, so C sends the values, not the places where they live.
`the printf after the call`	Prints main's own variables again. They are unchanged, which is the whole lesson of this question.
`int first = 11; int second = 22;`	The two originals, kept in main. They are the only variables the user cares about.
`the printf before the call`	Shows the starting values, so that the "after" line can be compared with something.
`int temp;`	A local variable of the function, used only for the swap. It disappears when the function ends.
`void, and no return`	Nothing is sent back, which is exactly why main never learns about the swap.
`the last printf`	States the lesson in words, so a student who ran the program knows what to look for.

OUTPUT

```
Before the call     : first = 11, second = 22
Inside the function : a = 22, b = 11
After the call      : first = 11, second = 22
The originals did not change - only the copies were swapped
```

Q5. Passing by Reference — Making the Originals Change

REAL SITUATION

The same two numbers, but this time the office must correct the record itself.

A photocopy is no use now. The office is told **where the originals are kept** — their addresses — so that it can reach them directly.

The swap is written exactly as before. Only two things change: & in the call, and the star in the function. That is enough to make the change real.

CONCEPT USED

Parameter passing by reference, using the address operator & from Assignment 4 and a pointer parameter `int *a`.

*a means "the value living at that address", so writing to it changes main's own variable.

Compare every line with Question 4 — the difference is only in those two places.

```c
#include <stdio.h>

void reallySwap(int *a, int *b) {   /* the two addresses arrive here */
    int temp;
    temp = *a;                      /* *a means the value at that address */
    *a = *b;
    *b = temp;
}

int main() {
    int first = 11;
    int second = 22;

    printf("Before the call : first = %d, second = %d\n", first, second);

    reallySwap(&first, &second);    /* the addresses are handed over */

    printf("After the call  : first = %d, second = %d\n", first, second);
    printf("This time the originals changed");

    return 0;
}
```

Question 4 and Question 5, side by side

	Q4 — BY VALUE	Q5 — BY REFERENCE
The call	`tryToSwap(first, second)`	`reallySwap(&first, &second)`
The parameters	`int a, int b`	`int *a, int *b`
Inside the swap	`temp = a;`	`temp = *a;`
What the function holds	two copies of the values	two addresses of main's boxes
Result in main	11 and 22 — unchanged	22 and 11 — really swapped

AT THE OFFICE	→	IN THE CODE
Telling them which locker holds the record	→	`&first`
The clerk noting that locker number down	→	`int *a`
Opening that locker and reading what is inside	→	`*a`
Putting a new value into that same locker	→	`*a = *b;`

What each side holds, with the addresses shown

11	22	2000	2004
first (main) address 2000	second (main) address 2004	a — holds an address	b — holds an address

Before the swap: the function holds two **locker numbers**, not two values

22	11	2000	2004
first — changed	second — changed	a — unchanged	b — unchanged

After the swap: the two addresses never moved. What changed is what lives **at** those addresses — main's own boxes.

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
`void reallySwap(int *a, int *b)`	The star in the declaration says each parameter holds an **address**, not a value.
`temp = *a;`	The star here means "the value at that address". Written without it, `temp = a;` would copy the address itself and the swap would fail.
`*a = *b; *b = temp;`	The same swap as always, but every read and write now reaches into main's own boxes.
`reallySwap(&first, &second);`	The & is what sends the addresses. Forgetting it here is the single most common mistake in this question.
`no printf inside`	None is needed. The two lines printed by main are enough to show that the originals themselves changed.
`int first = 11; int second = 22;`	The same two originals as in Question 4, so the two programs can be compared directly.
`int temp;`	Still an ordinary local int. Only the two parameters became pointers, not this one.
`void reallySwap(...)`	Still returns nothing — and it does not need to, because the change happens through the addresses.
`the printf before and after`	The two lines are the proof. In Question 4 they printed the same values; here they print different ones.
`the last printf`	States the result in words, matching the sentence in Question 4 that said the opposite.

OUTPUT

```
Before the call : first = 11, second = 22
After the call  : first = 22, second = 11
This time the originals changed
```

QUESTION 6 — PASSING AN ARRAY

Q6. Lab 5042 — Passing an Array of Marks to a Function

REAL SITUATION

Result day — five subject marks from Lab 5042: 78, 85, 62, 90 and 71.

The office needs the total and the average. Nobody carries five sheets across; the office is told **where the register is** and how many rows it has.

So the function receives the array (as an address) and the size, and hands back one number — the total.

CONCEPT USED

Passing an array and its size. An array always travels as an address, so **no &** is written in the call.

The size must be passed separately, because the function cannot work out how long the array is.

The loop inside is the plain `for` loop of Assignment 3 and the indexing of Assignment 4.

```c
#include <stdio.h>

int total(int marks[], int n) {     /* the array arrives as an address */
    int i;
    int sum = 0;
    for (i = 0; i < n; i++) {
        sum = sum + marks[i];
    }
    return sum;
}

int main() {
    int marks[5] = {78, 85, 62, 90, 71};
    int sum;

    sum = total(marks, 5);          /* no & is needed for an array */

    printf("Marks         : 78 85 62 90 71\n");
    printf("Total marks   = %d\n", sum);
    printf("Average marks = %d", sum / 5);

    return 0;
}
```

78	85	62	90	71
marks[0]	marks[1]	marks[2]	marks[3]	marks[4]

There is only **one** row of boxes. The function does not get a copy of it; it is told where this row begins.

AT THE OFFICE	→	IN THE CODE
Telling them where the register is kept	→	`total(marks, 5)`
Telling them how many rows it has	→	`the second argument, 5`
The clerk's own words for those two	→	`int marks[], int n`
Adding row after row	→	`sum = sum + marks[i];`
Handing back one number	→	`return sum;`

Step by step — the running total

`start`	→	sum = 0, before any mark is added
`i = 0, 1`	→	sum becomes 78, then 163
`i = 2, 3`	→	sum becomes 225, then 315
`i = 4`	→	**sum becomes 386 — the last mark has been added**
`i = 5`	→	5 < 5 is false, the loop ends, and 386 is returned

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
`int total(int marks[], int n)`	The empty brackets say "an array arrives here". No size is written inside them, because the function only receives an address.
`int sum = 0;`	The running total, started at 0. Starting from anything else would add a value that no student earned.
`for (i = 0; i < n; i++)`	Uses `n`, not 5, so the same function works for a class of any size.
`sum = sum + marks[i];`	Reads box `i` of main's own array — there is no second copy anywhere.
`total(marks, 5)`	The array name alone is already the address, so writing `&marks` here would be wrong.
`sum / 5`	Both sides are int, so C throws the decimal part away: 386 / 5 gives 77, not 77.2. A float average would need `sum / 5.0` and `%f`.
`int marks[5] = { ... };`	The five marks, kept in main. There is only one copy of this row anywhere in the program.
`int sum;`	A local variable of main, waiting for the value the function returns.
`int i;` inside the function	The loop counter is declared inside the function, so it belongs to the function alone.
`printf("Marks : ...")`	A fixed line, printed so the reader can add the numbers by hand and check the total.
`return 0;`	Belongs to main. The function has its own `return sum;`, which is a different thing.

OUTPUT

```
Marks         : 78 85 62 90 71
Total marks   = 386
Average marks = 77
```

Q7. Mess Bill — One Function Calling Another

REAL SITUATION

The billing clerk at the mess, and the rate counter behind him.

Three guest plates at Rs. 70 each, and rice for two of them at Rs. 10 extra.

You ask the billing clerk for the total. He asks the rate counter for the plate cost, adds the rice charge himself, and hands you the final figure. You never spoke to the rate counter.

CONCEPT USED

Nested calls — a function calling another function.

The rule of Concept 2 applies twice: each caller pauses, and each function finishes before the one that called it continues.

The Rs. 70 rule stays in **one** function, so it is still written only once.

```c
#include <stdio.h>

int plateCost(int plates) {                 /* the inner function */
    return plates * 70;
}

int billWithRice(int plates, int riceCount) {
    int cost;
    cost = plateCost(plates);               /* one function calling another */
    cost = cost + (riceCount * 10);
    return cost;
}

int main() {
    int bill;

    bill = billWithRice(3, 2);              /* main calls, which calls again */

    printf("Guest plates = 3 at Rs. 70 each\n");
    printf("Rice extra   = 2 at Rs. 10 each\n");
    printf("Total bill   = Rs. %d", bill);

    return 0;
}
```

Step by step — down two levels and back

`main`	→	calls billWithRice(3, 2) and pauses
`billWithRice`	→	**calls plateCost(3) and pauses in its turn**
`plateCost`	→	3 × 70 = 210 is returned, and this function ends first
`billWithRice`	→	continues: cost = 210, then 210 + 2 × 10 = 230, and returns 230
`main`	→	continues with bill = 230 and prints the three lines

AT THE MESS COUNTER	→	IN THE CODE
You asking the billing clerk for the total	→	`billWithRice(3, 2)`
He asking the rate counter for the plate cost	→	`plateCost(plates)`
Him adding the rice charge himself	→	`cost + (riceCount * 10)`
The final figure handed to you	→	`return cost;`

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
`int plateCost(int plates)`	Written **above** the function that uses it, so no prototype is needed. The rate rule lives here and nowhere else.
`cost = plateCost(plates);`	The nested call. The value that arrived in `billWithRice` is passed straight on to the inner function.
`cost = cost + (riceCount * 10);`	The rice charge is added afterwards. The brackets are not required by C, but they make the two parts of the bill clear to a reader.
`int cost;`	A local variable of `billWithRice`. Neither main nor plateCost can see it.
`bill = billWithRice(3, 2);`	Main makes one call and receives one number. It never needs to know that a second function was involved at all.
`int plates, int riceCount`	Two parameters, filled left to right from the call `billWithRice(3, 2)`.
`return plates * 70;`	The rate rule, living in `plateCost` only. Nothing else in the program knows the number 70.
`int bill;`	A local variable of main, holding the one number that comes back.
`the three printf lines`	Print the two inputs and the answer, so the arithmetic 3 × 70 + 2 × 10 can be checked by eye.
`order of the functions`	`plateCost` is written above `billWithRice`, so no prototype is needed for the inner call.

OUTPUT

```
Guest plates = 3 at Rs. 70 each
Rice extra   = 2 at Rs. 10 each
Total bill   = Rs. 230
```

Q8. Auditorium Register — the String Library Functions

REAL SITUATION

At the Auditorium register, two names are written in two columns: Subrata and Pramanik.

The clerk has to do four small jobs: **count the letters** of the first name, **copy** it into the full-name column, **join** the second name onto it, and **check** whether the two names are the same.

All four already exist as library functions in `string.h`, so nothing has to be written by hand this time.

CONCEPT USED

Library functions and the **header file** that declares them.

`#include <string.h>` is what makes `strlen`, `strcpy`, `strcat` and `strcmp` legal to call.

They all stop at the `'\0'` end mark of Assignment 4.

```c
#include <stdio.h>
#include <string.h>                     /* needed for the string functions */

int main() {
    char name1[20] = "Subrata";
    char name2[20] = "Pramanik";
    char full[40];

    printf("Name 1            : %s\n", name1);
    printf("Name 2            : %s\n", name2);

    printf("strlen(name1)     = %d\n", (int) strlen(name1));

    strcpy(full, name1);                    /* copy name1 into full */
    strcat(full, " ");                      /* add a space */
    strcat(full, name2);                    /* add name2 at the end */
    printf("After strcpy/strcat : %s\n", full);

    if (strcmp(name1, name2) == 0) {
        printf("strcmp says the two names are the same");
    } else {
        printf("strcmp says the two names are different");
    }

    return 0;
}
```

S	u	b	r	a	t	a	\0
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]

`strlen` counts boxes 0 to 6 and stops at the end mark, so the answer is **7** — the end mark is not counted

AT THE REGISTER DESK	→	IN THE CODE
Counting the letters of a name	→	`strlen(name1)`
Writing a name into the full-name column	→	`strcpy(full, name1);`
Adding the second name after it	→	`strcat(full, name2);`
Checking whether two names are the same	→	`strcmp(name1, name2) == 0`

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
`#include <string.h>`	Brings in the prototypes of the four functions. Without it the compiler does not know what `strlen` looks like and complains at the call.
`char full[40];`	Deliberately larger than both names together. `strcat` does not check the size, so making room is the programmer's job.
`(int) strlen(name1)`	`strlen` does not give a plain int, so `(int)` is written before printing it with `%d`. Without the cast the compiler gives a warning.
`strcpy(full, name1);`	Copies into **full** — the destination is always written first, exactly like an assignment.
`strcat(full, " ");`	Joins one space, so the two names do not run together as "SubrataPramanik".
`strcmp(name1, name2) == 0`	0 means the strings are **equal**. Writing `if (strcmp(...))` alone would mean "not zero", which is true when they are different.
`char name1[20], name2[20]`	Two character arrays, each with room for 19 letters and the end mark.
`the first two printf lines`	Print the names with `%s`, which stops at the end mark, exactly as in Assignment 4.
`if ... else`	The plain if-else of Assignment 2. Only the condition is new.
`the message inside else`	Runs because the two names really are different, which is what `strcmp` reported.
`return 0;`	Ends main. None of the four library functions needed a return value to be stored here.

OUTPUT

```
Name 1           : Subrata
Name 2           : Pramanik
strlen(name1)    = 7
After strcpy/strcat : Subrata Pramanik
strcmp says the two names are different
```

Q9. Campus Measurements — Macros and a Library Function

REAL SITUATION

Two measurements on the campus.

A **circular lawn** of radius 10 metres, whose area is needed; and a **square notice board** of side 5 metres, whose diagonal is needed.

The value of PI and the side of the board are fixed numbers that should be written in one place at the top, exactly like the rate on the mess board. And the square root is a job already done for us by the library.

CONCEPT USED

Macros with `#define`, and the library function `sqrt` from `math.h`.

A macro is replaced as plain text before compiling, so PI takes no memory and can never change.

Note that neither `#define` line ends with a semicolon.

```c
#include <stdio.h>
#include <math.h>                       /* needed for sqrt */

#define PI 3.14159                      /* a macro, not a variable */
#define SIDE 5

int main() {
    float area;
    float diagonal;

    area = PI * 10 * 10;                /* PI is replaced before compiling */
    printf("Circular lawn of radius 10 m\n");
    printf("Area = %.2f square metres\n\n", area);

    diagonal = sqrt(SIDE * SIDE + SIDE * SIDE);
    printf("Square notice board of side %d m\n", SIDE);
    printf("Diagonal = %.2f metres", diagonal);

    return 0;
}
```

WHAT YOU WRITE

```c
area = PI * 10 * 10;
diagonal = sqrt(SIDE * SIDE + SIDE * SIDE);
```

WHAT THE COMPILER IS GIVEN

```c
area = 3.14159 * 10 * 10;
diagonal = sqrt(5 * 5 + 5 * 5);
```

The replacement is plain text and happens before compiling — this is why a macro has no type and takes no memory

ON THE CAMPUS	→	IN THE CODE
The value of PI, written once at the top	→	`#define PI 3.14159`
The side of the notice board, written once	→	`#define SIDE 5`
The area of the circular lawn	→	`area = PI * 10 * 10;`
The square root, a job already done for us	→	`sqrt(...) from math.h`
Printing a measurement to two decimals	→	`%.2f`

What the two macros become

PI	3.14159	SIDE	5
what you write	what the compiler sees	what you write	what the compiler sees

The replacement happens **before** compiling, as plain text. Nothing is stored in memory, and neither name can ever change while the program runs.

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
`#include <math.h>`	Brings in the prototype of `sqrt`. On Linux the program is compiled with `gcc file.c -lm`; inside VS Code on Windows this is usually not needed.
`#define PI 3.14159`	A macro. **No semicolon** — writing one would put it into the replacement and break every line that uses PI.
`#define SIDE 5`	Used twice in the calculation and once in the printf, so the side of the board is written in one place only.
`area = PI * 10 * 10;`	The formula πr^2 with r = 10. After replacement this is an ordinary multiplication of numbers.
`sqrt(SIDE * SIDE + SIDE * SIDE)`	The diagonal of a square: $\sqrt{(5^2 + 5^2)} = \sqrt{50} = 7.07$. `sqrt` is a library function, already written and tested.
`%.2f`	Prints a decimal number with exactly two digits after the point, so 314.159 appears as 314.16.
`%d with SIDE`	SIDE is replaced by the plain number 5, so `%d` is correct here.
`float area; float diagonal;`	Two local variables of main. `float` is used because both answers have a decimal part.
`the printf heading lines`	Fixed lines naming what is being measured, so each number on the screen has a label.
`\n\n at the end of one printf`	Leaves one blank line, so the lawn result and the notice-board result do not run together.

OUTPUT

```
Circular lawn of radius 10 m
Area = 314.16 square metres

Square notice board of side 5 m
Diagonal = 7.07 metres
```

QUESTION 10 — THE STATIC VARIABLE

Q10. Main Gate Register — a static Variable that Remembers

REAL SITUATION

The register at the IIITA Main Gate.

Three students enter one after another. For each of them the guard does the same thing, so one function is called three times.

But the guard keeps **two** things: a register that stays in the drawer and remembers every entry so far, and a rough slip that he starts fresh for each visitor and throws away afterwards.

The program shows both, side by side, so the difference is impossible to miss.

CONCEPT USED

Storage classes: a `static` local variable against a plain (auto) one.

The static one is created **once** and keeps its value between calls; the plain one is created fresh at every call.

Both are still **local** — main cannot see either of them.

```c
#include <stdio.h>

void enterGate() {
    static int count = 0;              /* created once, remembered for ever */
    int today = 0;                     /* created fresh at every call */

    count = count + 1;
    today = today + 1;

    printf("Entry %d : static count = %d, plain variable = %d\n", count, count, today);
}

int main() {
    printf("Main gate register\n");

    enterGate();
    enterGate();
    enterGate();

    printf("\nThe static variable kept counting, the plain one started again every time");

    return 0;
}
```

Step by step — the two variables over three calls

`call 1`	→	count is created once and becomes 1; today is created and becomes 1
`call 1 ends`	→	today is destroyed; count stays alive in memory
`call 2`	→	**count is not created again — it still holds 1, so it becomes 2; today is created fresh and becomes 1**
`call 3`	→	**count becomes 3; today is 1 once more**
`the lesson`	→	the static variable counted the entries; the plain one could not, because it forgot everything at each call

AT THE GATE	→	IN THE CODE
The register kept in the drawer	→	`static int count = 0;`
The rough slip used for one visitor	→	`int today = 0;`
One student entering	→	`enterGate();`
Adding one line to the register	→	`count = count + 1;`

The two variables over the three calls

1	2	3
count — call 1	count — call 2	count — call 3

`static int count` — created once, so every call adds to what was already there

1	1	1
today — call 1	today — call 2	today — call 3

`int today` — created fresh at every call, so it can never get past 1

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
`static int count = 0;`	The `= 0` runs **only at the first call**. On later calls this line is skipped and the old value is kept — that is what makes counting possible.
`int today = 0;`	An ordinary local variable. Its `= 0` runs at **every** call, which is why it can never get past 1.
`count = count + 1;`	Adds one entry to the register. Written as `count++` it would mean exactly the same thing.
`printf(..., count, count, today);`	Three values are printed, so three `%d` are written. The first two use the same variable on purpose: the entry number and the register total are the same thing.
`three calls in main`	The same function is used three times without being written three times — the whole point of Concept 1.
`count is still local`	Although it lives as long as the program, main cannot read or change it. That is what separates `static` from a global variable.
`void enterGate()`	No parameter and no return value — the function only records an entry, exactly like Question 1.
`the two declarations together`	Written one under the other on purpose, so the only difference on the page is the word `static`.
`today = today + 1;`	Runs at every call, but always on a brand-new variable, so the answer is always 1.
`printf("Main gate register\n");`	A heading printed once by main, before any entry is recorded.
`the last printf`	Says in words what the three output lines show, so the lesson is not left to be guessed.

OUTPUT

```
Main gate register
Entry 1 : static count = 1, plain variable = 1
Entry 2 : static count = 2, plain variable = 1
Entry 3 : static count = 3, plain variable = 1

The static variable kept counting, the plain one started again every time
```

Quick Revision — Concept, Real IIITA Memory, Code Shape

CONCEPT	REMEMBER IT AS	CODE SHAPE
What a function is	the photocopy counter: hand something over, a fixed job is done, something comes back	`int name(int x) { ... }`
Calling	standing at the counter and waiting; the caller pauses until the job is done	`total = guestBill(3);`
Parameter and argument	"three plates" is the argument; the clerk's word for it is the parameter	`f(3) → int f(int plates)`
return	what is handed back across the counter; it ends the function at once	`return bill;`
void	the notice board: the job is done, but nothing is handed back	`void printMenu()`
Prototype	the board above the counter, announcing the job in advance	`int f(int a, int b);`
Local variable	the clerk's rough sheet, thrown away at the end	`declared inside the function`
Global variable	the corridor notice board that every function can read	`declared outside every function`
By value	you hand over a photocopy, so your original is safe	`f(first) → int a`
By reference	you give the locker number, so the original itself can be changed	`f(&first) → int *a`
Passing an array	you do not carry the register, you say where it is	`f(marks, 5) → int marks[], int n`
Nested call	one clerk asking another clerk; the inner one finishes first	`a function calling a function`
Library function	the machines that were already installed on campus	`printf, strlen, sqrt`
Header file	the board that tells the compiler what those machines accept	`#include <string.h>`
Macro	the rate painted on one board and read everywhere	`#define PI 3.14159`
static	the register kept in the drawer, which remembers	`static int count = 0;`
auto, register, extern	the rough sheet, the number kept in the head, the register of another office	`int x; register int i; extern int r;`

— End of Assignment 5 —